GCA Project: BarraCuda

Feniks De Smet & Thomas Groothuis

April 2026

1 Introduction

BarraCuda is a marine ecosystem simulation involving three primary entities: Seaweed, Fish, and Sharks. The simulation allows users to configure initial population counts and the spawn frequency of seaweed (only in the pygame-cpu version). The code can be found on the github page:
https://github.com/Thirithos/BarraCuda

- **Seaweed:** Periodically spawns and serves as the base food source. It shrinks in size as it is consumed by fish.
- **Fish:** Entities that must consume seaweed to survive, failure to eat results in death. Adults can mate to produce offspring and sustain the population. The fish school using a Boids algorithm implementation.
- **Sharks:** Predators that follow logic similar to fish but hunt fish as their primary food source.

2 CPU Implementation

The baseline implementation of the BarraCuda simulation is developed in Python, utilizing an Object-Oriented Programming (OOP) architecture. While this design provides clarity and ease of development for small-scale models, it introduces significant performance bottlenecks that prevent the simulation from scaling to large populations. Developing OOP in python is also a more difficult task.

2.1 Software Architecture

The CPU-based source code is organized into distinct logical modules:

- **entities.py:** This module defines the core agents of the ecosystem: fish, sharks, and seaweed as individual classes. Each entity is an instance of a class that maintains its own state, including position, velocity, and energy levels.

- **simulation.py:** This file contains the primary ecosystem logic. It calculates entity interactions, such as fish seeking seaweed for food and sharks hunting fish.
- **main.py:** Functions as the simulation driver, responsible for the execution of the main loop, state synchronization, and the graphical representation of entity data using the Pygame library.

2.2 Computational Bottlenecks

During the performance analysis of the CPU version, three fundamental architectural hurdles were identified that necessitate a hardware-accelerated approach.

2.2.1 Serial Execution

The simulation is inherently complex because every entity must be aware of its surroundings to make decisions. In the Python implementation, this is handled via nested loops. For a population of N fish and M seaweed entities, the CPU must perform $N \times M$ distance checks per frame. Because these calculations are processed serially (one by one), the compute time grows quadratically, causing the framerate to collapse as the population increases.

2.2.2 Memory Fragmentation and Pointer Chasing

In Python's OOP model, every entity defined in *entities.py* is a separate object allocated at various locations in RAM. These objects are not stored contiguously in physical memory. As the CPU iterates through the list of fish, it must constantly jump between different memory addresses. This phenomenon, known as *Pointer Chasing*, results in high cache miss rates. The CPU spends a significant portion of its execution time waiting for data to arrive from the relatively slow main memory instead of performing actual calculations.

2.2.3 Python Interpreter and Rendering Overhead

Python introduces execution overhead for every operation, such as vector addition or conditional checks. Furthermore, the Global Interpreter Lock (GIL) prevents the simulation from effectively utilizing multiple CPU cores for parallel computation. Consequently, the entire ecosystem logic is restricted to the performance of a single execution thread, further increasing the serial execution bottleneck. Finally, relying on the CPU to calculate the simulation logic *and* issue draw calls for thousands of individual sprites via Pygame creates a significant rendering bottleneck. This creates a severe data transfer penalty across the system bus, highly motivating a transition to a hardware-accelerated rendering pipeline where simulation data remains on the GPU.

3 C++ Optimizations and GPU Architecture

3.1 Structure of Arrays (SoA)

In the GPU version we used a data-oriented design called Structure of Arrays (SoA). Instead of having an array of structures (e.g., $[x_1, y_1, z_1], [x_2, y_2, z_2]$) which is slower to access due to cache pollution, we use flat arrays for each attribute (e.g., $[x_1, x_2, x_3], [y_1, y_2, y_3]$). This contiguous memory layout ensures high cache coherency, meaning the processor fills its cache lines entirely with relevant data, drastically maximizing memory throughput.

3.2 O(1) Garbage Collection

Another great feature is the garbage collection of dead fish. By swapping the data of the latest fish in the array to the location of the dead fish and deallocating the last index of the array, dead fish can be removed in O(1) instead of O(N).

3.3 Memory Safe Spawning (Preventing Pointer Invalidation)

During the biological lifecycle update phase, entities can reproduce and spawn offspring. In standard C++, dynamically appending elements (`push_back`) to a `std::vector` while concurrently iterating over it is highly dangerous. If the vector exceeds its current capacity, it reallocates its data to a new heap location, instantly invalidating the active iterators and causing segmentation faults.

To guarantee memory safety without introducing expensive locking mechanisms, BarraCuda implements a *Deferred Spawning Buffer*. Offspring data (position, velocity, type) generated during the evaluation loop is temporarily pushed to local, contiguous buffers. Only after the primary iteration is completely finished, the buffered entities are flushed and safely instantiated into the primary SoA structure.

3.4 Algorithmic Optimization: The Spatial Grid

The `SpatialGrid` data structure was implemented to resolve the $O(N^2)$ algorithmic bottleneck present in the baseline simulation. In the original approach, every entity had to calculate its distance to every other entity to determine interactions (such as Boids flocking or predator evasion). The `SpatialGrid` divides the simulation space into discrete, uniform cells based on the maximum vision radius of the enitities. An entity only queries its own cell and the eight immediate neighboring cells. This algorithmic optimization effectively degrades the interaction complexity from quadratic $O(N^2)$ to near-linear $O(N)$, saving millions of CPU cycles per frame.

3.4.1 Architectural Tradeoff: CPU vs. GPU Partitioning

Although spatial partitioning is a technique frequently offloaded to GPU Compute Shaders, the grid logic in BarraCuda was deliberately retained on the CPU. This architectural tradeoff was made to maximize hardware synergy and prevent systemic bottlenecks:

- **PCIe Bus Latency:** Because the core kinematic and biological physics are processed on the CPU utilizing the SoA layout, offloading the grid construction to the GPU would require constant bi-directional data transfers. The CPU would have to send the ecosystem state to the GPU, wait for the grid construction, and pull the binned data back across the PCIe bus to compute physics. The latency of these memory transfers would completely negate any computational speedup.
- **Warp Divergence:** The GPU SIMT (Single Instruction, Multiple Threads) architecture demands that threads within a warp execute the same instructions. Querying a spatial grid involves highly variable loop counts. One cell might contain zero neighbors, while another contains forty. On a GPU, this causes massive *Warp Divergence*, leaving cores idle while they wait for the longest loop to finish. The CPU, equipped with advanced branch prediction and deep L1/L2 caches, handles these divergent, localized spatial loops far more efficiently.

3.5 Hardware-Accelerated Instanced Rendering

By dedicating the CPU to the $O(N)$ logic and physics, the architecture was perfectly positioned to offload the final bottleneck to the GPU: the rendering overhead. Instead of issuing an individual draw call to the operating system for every single entity, which heavily saturates the PCIe bus, the simulation utilizes OpenGL *Instanced Rendering*.

The base geometry of an entity (a simple 2D quad consisting of four vertices) is loaded into the GPU's VRAM via a Vertex Buffer Object (VBO) only once during initialization. Every frame, the CPU gathers the dynamic state from the SoA ecosystem and packs it into a flat array of `InstanceData` structs (containing position offsets, rotations, and scales). By utilizing `glVertexAttribDivisor`, the GPU's input assembler is instructed to reuse the static base geometry, applying the unique transformation data to each instance. This allows the simulation to render tens of thousands of entities using a single *Draw Call* (`glDrawElementsInstanced`). To fully grasp the efficiency of this pipeline, a deeper examination of the memory streaming protocols and custom shader mechanics is required.

3.5.1 Low-Level Memory Mapping and Data Streaming

To stream the `InstanceData` to the GPU efficiently, the pipeline utilizes `GL_DYNAMIC_DRAW`. Because the data changes every frame, this flag hints the

graphics driver to place the buffer in a memory location optimized for frequent CPU writes and fast GPU reads.

Furthermore, the architecture relies on strict memory mapping using the C++ `offsetof` macro. When configuring the Vertex Attribute Pointers, the CPU explicitly tells the OpenGL state machine the exact byte offset of the position, rotation, and scale within the tightly packed C++ struct. This direct memory mapping ensures zero parsing overhead when the GPU pulls the array from VRAM into the shader pipeline.

3.5.2 SIMT Math Offloading in the Vertex Shader

Offloading of matrix mathematics is another performance increase. In traditional architectures, the CPU calculates the transformation matrices for sprites. In BarraCuda, the CPU simply passes a single float (the rotation angle in radians) via the instance buffer.

The GPU's **Vertex Shader** leverages its SIMT (Single Instruction, Multiple Threads) architecture to process the trigonometry massively in parallel across thousands of cores:

```
float c = cos(iRot);
float s = sin(iRot);
mat2 rotMat = mat2(c, s, -s, c);
vec2 pos = (rotMat * (aPos * iScale)) + iOffset;
gl_Position = projection * vec4(pos, 0.0, 1.0);
```

Here, the GPU natively calculates the sine and cosine, constructs the 2D rotation matrix (`rotMat`), scales the base vertex (`aPos`), and applies the translation (`iOffset`). This frees up significant CPU clock cycles that are instead reinvested into the core physics simulation.

3.5.3 Implicit Geometry via Fragment Culling (The Discard Trick)

When the debug mode is activated, the simulation visualizes the exact collision radius (hitbox) of each entity. Rendering true circles typically requires loading high-polygon meshes (e.g., a circle made of 32 triangles), which massively increases vertex processing overhead. To bypass this, BarraCuda utilizes *Implicit Geometry* in the **Fragment Shader**:

```
if(isD) {
    if(distance(TexCoord, vec2(0.5, 0.5)) > 0.5) discard;
    FragColor = col;
}
```

Instead of passing circular geometry, the pipeline reuses the same simple, 4-vertex quad. During the rasterization phase, the Fragment Shader calculates

the mathematical distance from the current pixel to the center of the texture coordinate. If the pixel falls outside the normalized radius of 0.5, the `discard` command is called. The GPU immediately aborts processing that pixel, effectively carving a perfect mathematical circle out of a square quad. This technique provides pixel-perfect debug spheres with zero additional geometry overhead, proving the extreme flexibility of the programmable graphics pipeline.

4 Performance Evaluation & Benchmarks

To empirically validate the architectural optimizations, both the Python baseline and the C++/OpenGL pipeline were subjected to automated benchmarking.

4.1 Methodology

A benchmark sequence was integrated directly into the main execution loop. The simulation was configured to run continuously for 10 minutes (600 seconds), starting with a small initial population. Every second, the system aggregates the total number of active entities (Fish, Sharks, and Plants) and logs it against the current rendering throughput (Frames Per Second or FPS). This data was written to CSV files (`benchmark_cpu.csv` and `benchmark_gpu.csv`) to visualize scalability under increasing ecosystem loads.

4.2 Scalability Comparison

A benchmark sequence was integrated directly into the main execution loop. The simulation was configured to run continuously for 10 minutes (600 seconds), starting with a small initial population. Every second, the system aggregates the total number of active entities (Fish, Sharks, and Plants) and logs it against the current rendering throughput (Frames Per Second or FPS). This data was written to CSV files to visualize scalability under increasing ecosystem loads.

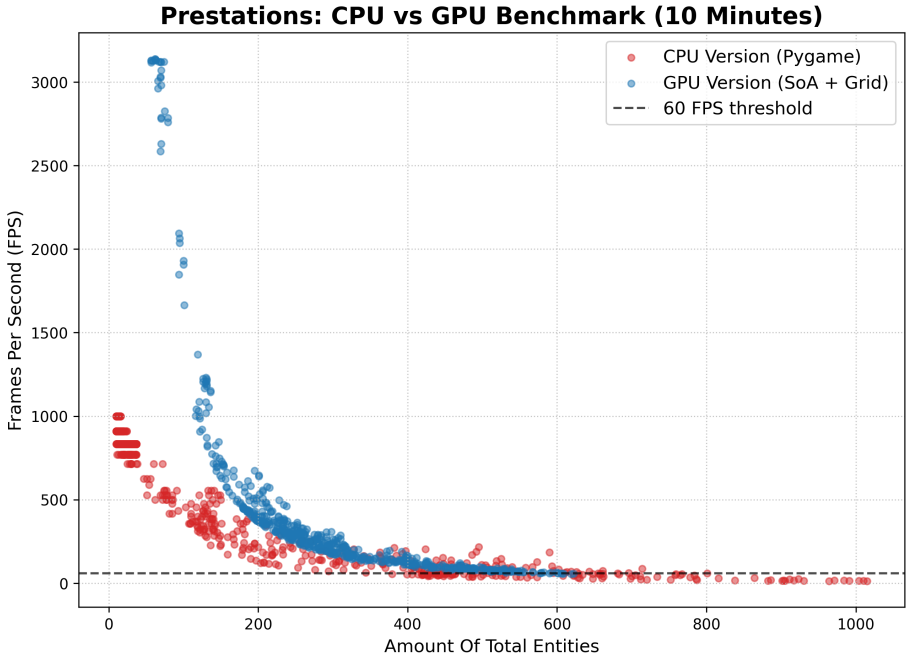


Figure 1: Macro-level performance scaling: CPU (Pygame) versus the final optimized GPU architecture (SoA + Spatial Grid).

As visualized in Figure 1, the overall scalability dramatically improved after migrating from Python to C++ and applying Data-Oriented Design alongside OpenGL Instanced Rendering. The Python CPU implementation suffers a rapid, exponential performance degradation. Due to its $\mathcal{O}(N^2)$ algorithmic complexity and interpreter overhead, the CPU's framerate drops below the acceptable threshold of 60 FPS at approximately 400 entities.

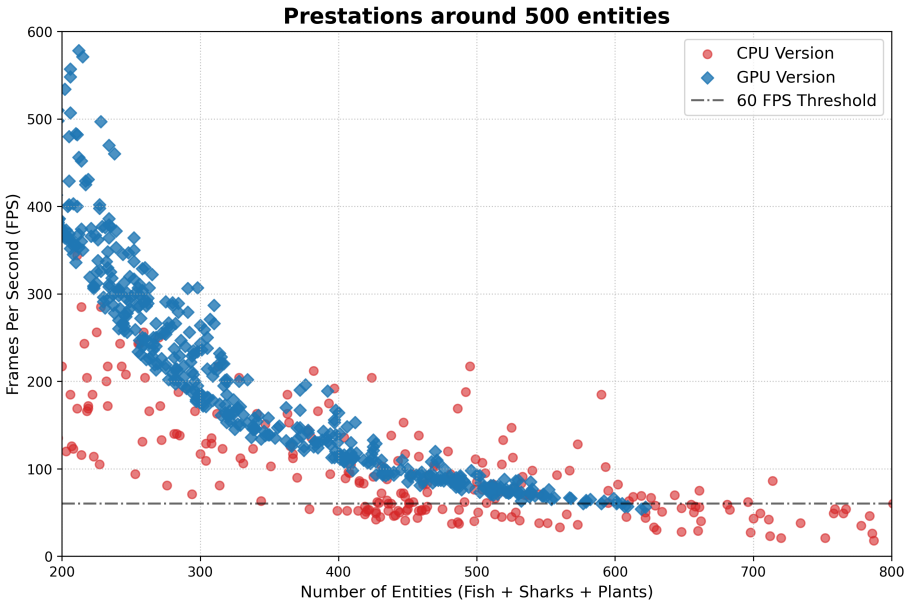


Figure 2: Detailed view of the 200-800 entity range, highlighting the exact point where the CPU version becomes unplayable, while the GPU version remains highly stable.

Figure 2 provides a closer look at the critical 200-800 entity range. It clearly illustrates the stability of the ultimate GPU architecture (Structure of Arrays combined with a Spatial Grid), which absorbs the ecosystem growth with a much shallower performance decay compared to the steep collapse of the CPU baseline.

However, an even deeper analysis comparing all intermediate optimization phases reveals a fascinating architectural trade-off at these lower entity counts, as detailed in Figure 3.

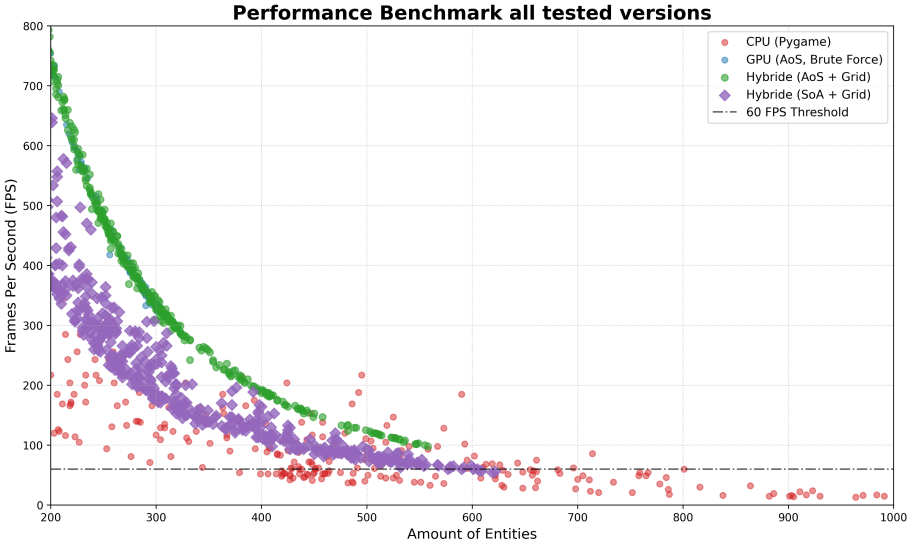


Figure 3: Performance comparison of all four optimization phases. Note the crossover point where the simpler AoS Brute Force method temporarily outperforms the advanced SoA Grid architecture.

As shown in the multi-version benchmark comparison (Figure 3), the simpler Array of Structures (AoS) brute-force $\mathcal{O}(N^2)$ approach actually outperforms the optimized Structure of Arrays (SoA) combined with a Spatial Grid when the population is under 400 entities. This counterintuitive result perfectly illustrates the concept of algorithmic overhead.

At smaller population sizes, the computational cost of dynamically allocating, clearing, and querying the spatial grid every single frame introduces a constant overhead that outweighs the benefits of reduced distance checks. Modern CPUs are incredibly efficient at executing tight, predictable loops (like a brute-force distance check) when the entire dataset is small enough to fit neatly into the ultra-fast L1/L2 CPU cache.

However, this brute-force supremacy is strictly short-lived. The moment the ecosystem population grows beyond this crossover point, the exponential penalty of the $\mathcal{O}(N^2)$ algorithm rapidly collapses the framerate. In contrast, the $\mathcal{O}(N)$ complexity and memory-contiguous layout of the SoA grid architecture effortlessly absorb the growing entity count, maintaining a stable and playable simulation long after the brute-force method has failed.

5 Shortcomings and Future Improvements

While the transition to a C++ Structure of Arrays and OpenGL Instanced Rendering yielded massive performance gains, a critical architectural review of the codebase reveals several remaining bottlenecks.

5.1 Standard Library Overhead: The `std::vector<bool>` Trap

In the `Ecosystem` SoA definition, entity flags such as `is_alive` and `is_pregnant` are stored using `std::vector<bool>`. In the C++ standard, this specific container is heavily optimized for space by packing 8 booleans into a single 1-byte bitfield. Because the CPU can only address memory at the byte level, accessing a single flag forces the processor to execute hidden bitwise operations (AND, OR, bit-shifting). This breaks cache coherency and introduces unnecessary ALU instructions. A direct improvement would be replacing these vectors with `std::vector<uint8_t>`, sacrificing a trivial amount of RAM to guarantee rapid, byte-aligned memory access.

5.2 Heap Fragmentation in the Spatial Grid

The `SpatialGrid` successfully resolves the $O(N^2)$ algorithmic bottleneck, but its underlying data structure introduces a memory management flaw. The grid is implemented as a 2D vector: `std::vector<std::vector<int>>`. While the outer vector allocates its control blocks contiguously, the inner vectors dynamically allocate their data at random locations on the heap. When an entity queries its neighboring cells, the CPU performs pointer chasing across fragmented heap memory, causing L1/L2 cache misses. A high-performance implementation should flatten the grid into a single, contiguous 1D array to ensure strict spatial locality.

5.3 Memory Bandwidth Saturation (Loop Fusion)

Currently, the main simulation loop is constrained by memory bandwidth. Throughout a single frame, the CPU iterates over the entire population multiple times in separate passes (lifecycle updates, grid insertion, physics calculation, and rendering data packaging). Fetching the same massive arrays from the main RAM to the CPU registers multiple times per frame is highly inefficient. Future improvements should focus on **Loop Fusion**. By combining these passes, an entity's data can be loaded into the CPU's fast L1 cache once, updated, and written to the OpenGL `InstanceData` buffer before being evicted from the cache.

6 Conclusion

The BarraCuda simulation successfully demonstrates the necessity of hardware-aware programming. By abandoning object-oriented overhead in favor of Data-Oriented Design, and by deliberately dividing the workload, assigning complex $O(N)$ spatial logic to the CPU while offloading parallel matrix transformations and rendering to the GPU, the project achieved immense scalability.